

Project Report DevOps

Adrian Grund
agrund@kth.se

Alexander Runebou
alerun@kth.se

October 10, 2026

1 Project Overview and Objectives

Our project, **Scheduler**, is a student scheduling application in which users can form groups and coordinate meetings. The purpose of the DevOps work was not to maximize application functionality, but rather to build a coherent workflow that takes changes from a feature branch to a running deployment with automated verification, controlled promotion, and operational feedback.

The application consists of a FastAPI/Python backend, a TypeScript frontend, a background worker, and a PostgreSQL database. Schema changes are handled using Alembic and package services as Docker images. We use GitHub for source control, pull requests, branch protection, and GitHub Actions workflows. The development deployment runs on a Raspberry Pi with a self-hosted GitHub Actions runner, while the production deployment runs on a single-node k3s Kubernetes cluster on an Azure virtual machine. Deployment therefore runs on real machines rather than stopping at just a successful build.

2 Architecture and End-to-End Workflow

2.1 Development

Figure 1 shows the main promotion path. Developers work on feature branches and open pull requests against the protected `develop` branch. GitHub-hosted runners execute backend, frontend, and security checks. A merge has to pass the required status checks and receive at least one approval before being accepted. Once merged, a separate deployment pipeline builds and publishes an image and resolves its SHA digest. Before it gets deployed to the Raspberry Pi, we require an approval from us as a prevention against malicious code. The self-hosted runner then pulls the images, runs migrations, starts the services, and performs health checks.

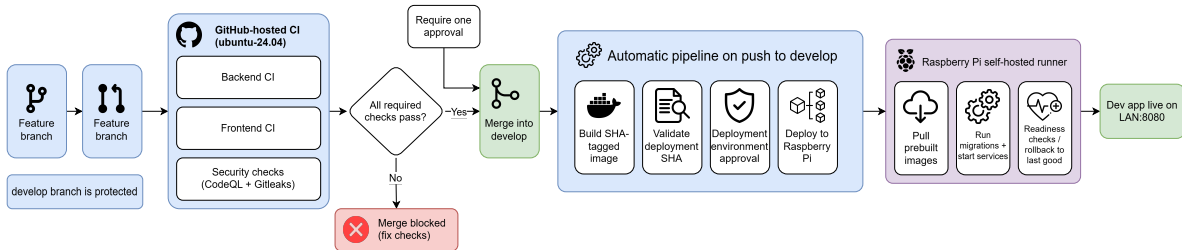


Figure 1: Integrated CI/CD workflow from feature branch to the Raspberry Pi development environment.

We have deliberately designed this around *separation of concerns*. CI validates a proposed change, branch protection governs promotion into `develop`, and CD deploys an approved revision. This makes it easier to detect failures and prevent an unverified pull request from directly changing the deployed environment.

2.2 Production

Production is promoted from `main` (Figure 2). A push builds both images for `linux/amd64`, scans them with Trivy and publishes them to GHCR only if no fixable critical vulnerability is found. The tag `main-<sha>` keeps them apart from the ARM images built for the Raspberry Pi.

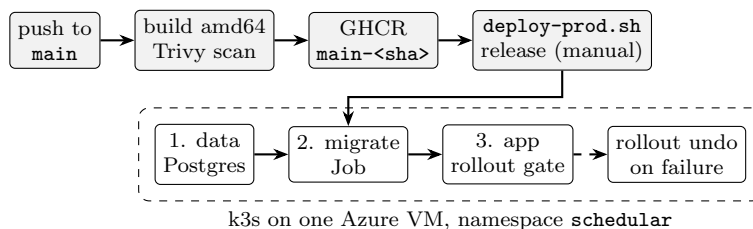


Figure 2: Production path from `main` to the k3s cluster on Azure.

In the cluster, Traefik routes `/api` to the backend and everything else to the frontend under one hostname, so the browser sees a single origin; cert-manager obtains Let’s Encrypt certificates. Mail goes to an internal Mailpit, reachable only through an SSH tunnel.

3 Implementation of Required DevOps Practices

3.1 Automated Build and Testing (CI)

We use GitHub Actions to run checks for both major application components. Backend CI uses a PostgreSQL service and executes automated tests, and code-quality checks include Ruff and mypy. Frontend CI provides a separate validation path so failures guide us to the correct component. Container images are built by an image-building workflow after code is promoted.

3.2 Quality and Security Automation

The pipeline uses CodeQL static analysis and Gitleaks secret scanning as automated security checks. Together with linting, type checking, and tests, these provide coverage against breaking coding conventions and likely defects, bad security patterns, accidentally committed secrets, and functional regressions. Since we use protected branches and require checks, these analyses are enforceable merge conditions rather than just optional local conventions. But it’s still important to remember that these tools only reduce risk, rather than prove the application is free of vulnerabilities.

On the production VM only ports 22, 80 and 443 are open, SSH is key-only, and the Kubernetes API is reached solely through an SSH tunnel.

3.3 Continuous Deployment and Rollback

Changes merged into `develop` trigger the deployment path depicted in Figure 1. Images are identified by a SHA-based reference, rather than a mutable tag such as `latest`, to make the deployed artifact easily identifiable. The self-hosted runner uses Docker Compose to pull and start the application stack. Alembic migrations are executed as a distinct deployment step, followed by service readiness checks using the backend’s liveness and database-aware readiness endpoints (`/healthz` and `/readyz`). A last-known-good image reference is always kept to support application rollback after a failed deployment. We also investigated migration-failure scenarios.

Production releases run through one script, `deploy-prod.sh`, in three separately applied lifecycles: data (a PostgreSQL StatefulSet), migration (a Job running `alembic upgrade head`) and application. A single `kubectl apply` would race new code against the migration. The script refuses a dirty tree or a commit that differs from the image tag, stops if the migration fails, gates on `rollout status` and runs `rollout undo` on failure. The database is never downgraded, so migrations must stay compatible with the previous release (expand/contract).

Production releases are started by hand on purpose. To conserve the student credit, the VM is deallocated whenever it is not in use, so an automatic deployment would either fail or force the VM

to run continuously. A workflow running the same script after approval in a protected `production` environment is prepared but not activated.

3.4 Infrastructure as Code

All infrastructure is version-controlled: Docker Compose files, Dockerfiles and workflow YAML for development, Kustomize bases and overlays for the Kubernetes manifests, Helm with pinned chart versions and committed values for Traefik and cert-manager, and Bicep for the Azure VM, network and firewall rules, previewed with `az deployment what-if` before anything billable is created. The VM installs a pinned k3s through cloud-init. Secrets are created out of band and referenced by name, never committed. Every manifest was first rehearsed on a local `kind` cluster with the same overlays, at no cost.

4 Design Decisions, Trade-offs, and Limitations

We selected GitHub Actions because it integrates directly with pull requests, branch rules, and repository-hosted workflows. Using GitHub-hosted runners for CI keeps verification isolated from the deployment machine; using a self-hosted runner for CD allows the deployment steps to run where the application is hosted. The trade-off is increased operational responsibility: we must maintain the runner, Docker installation, host credentials, storage, and network access ourselves.

For production, the constraint was a student credit. AKS would cost about USD 63 per month running and USD 32 stopped, since its load balancer and disks keep billing. k3s on one VM costs about USD 38 and USD 6 deallocated. We chose k3s, accepted operating the node ourselves, and control cost by deallocating the VM and releasing manually. Splitting `/api` at the ingress gives a cleaner traffic path than proxying through the frontend's nginx, but differs from the Compose stack. Limitations: a single node, no automated database backups, mail only to an internal sink, and manually triggered production releases.

5 AI-Assisted Development and Reflection

AI-assisted tools were mostly used for developing the application and relevant unit tests, but we also used it during development for technical discussion and implementation support, such as exploring CI/CD configuration, debugging workflow failures, and reviewing code. Generated suggestions were treated as proposals that had to be reviewed, tested, and verified against the running system.

Logs of agentic AI usage primarily during frontend creation can be found in the repository.

The main outcome is an integrated feedback-to-deployment loop: proposed changes are reviewed and tested before promotion; approved revisions are built and deployed using versioned configuration; and deployment success is assessed through migrations and health checks. The most important lesson is that a successful CI run is not equivalent to a successful deployment. Real deployment requires coordinating artifacts, database state, host availability, and application readiness. Our solution demonstrates these interactions while making its single-host and database-recovery limitations explicit.